

---

## Bayesian Optimization Algorithm, Decision Graphs, and Occam's Razor

---

**Martin Pelikan**

Dept. of General Engineering  
and Dept. of Computer Science  
University of Illinois, Urbana, IL 61801  
pelikan@illigal.ge.uiuc.edu

**David E. Goldberg**

Dept. of General Engineering  
University of Illinois  
Urbana, IL 61801  
deg@uiuc.edu

**Kumara Sastry**

Dept. of General Engineering  
University of Illinois  
Urbana, IL 61801  
kumara@illigal.ge.uiuc.edu

### Abstract

This paper discusses the use of various scoring metrics in the Bayesian optimization algorithm (BOA) which uses Bayesian networks to model promising solutions and generate the new ones. The use of decision graphs in Bayesian networks to improve the performance of the BOA is proposed. To favor simple models, a complexity measure is incorporated into the Bayesian-Dirichlet metric for Bayesian networks with decision graphs. The presented modifications are compared on a number of interesting problems.

## 1 INTRODUCTION

Recently, the use of local structures, such as default tables and decision trees/graphs, in context of learning the structure of Bayesian networks has been proposed and discussed (Friedman & Goldszmidt, 1999; Chickering, Heckerman, & Meek, 1997). Using local structures has shown to improve the performance of learning in terms of the likelihood of the resulting models on a number of benchmark data sets. However, none of these approaches was used to improve model building in the Bayesian optimization algorithm (Pelikan, Goldberg, & Cantú-Paz, 1998), which uses Bayesian networks to model promising solutions and guide the search. Moreover, the use of various metrics in the BOA has not been investigated thoroughly.

The purpose of the paper is twofold. First, the use of decision graphs in the model construction phase of the BOA is proposed to improve its performance. Second, to eliminate superfluously complex models, a model complexity measure is incorporated into the Bayesian-Dirichlet scoring metric for Bayesian networks with decision graphs. It is empirically shown that the in-

troduced pressure is sufficient to eliminate the necessity of binding the complexity of models by the user. This is a significant contribution. The advantages of the minimum description length (MDL) metric, which favors simple models, are attained without having to sacrifice a possibility of using prior knowledge about the solved problem introduced by Bayesian metrics. The performances of the BOA with various scoring metrics and network construction algorithms, and the simple genetic algorithm are compared on a number of problems.

The paper starts by describing the BOA. Section 3 provides basic theoretical background of learning a structure of Bayesian networks. Section 4 describes how decision graphs can be used as a core component of learning the structure of Bayesian networks and provides a Bayesian scoring metric for computing the marginal likelihood of Bayesian networks using decision graphs given the data. The results of our experiments are described in Section 5. The paper is concluded in Section 6.

## 2 BAYESIAN OPTIMIZATION ALGORITHM

It has been shown that using recombination and selection is a very powerful approach for optimizing many difficult problems. However, fixed, problem independent, recombination and mutation operators often result in inferior performance even on simple problems. Without knowing where the important interactions in the problem are and designing problem specific operators that take this information into account, the number of fitness evaluations and the required population sizes grow exponentially with the number of decision variables (Thierens & Goldberg, 1993).

Much effort was put in the design of methods that would be able to learn which parts of the solutions should be combined and which ones should remain

intact. One of the approaches replaces traditional crossover and mutation by building a probabilistic model of promising solutions and using this model to generate offspring. Probability distributions can capture variables which are correlated and the ones which are independent. This can subsequently be used to combine the solutions in more effective manner. Methods based on this principle are called estimation of distribution algorithms (Mühlenbein & Paaß, 1996), probabilistic model-building genetic algorithms (PMBGAs) (Pelikan, Goldberg, & Lobo, 2000), or iterated density estimation algorithms (Bosman & Thierens, 2000).

It is beyond the scope of this paper to give a complete overview of PMBGAs and the interested reader should refer to Pelikan et al. (2000). In this paper we focus on the Bayesian optimization algorithm (BOA) (Pelikan, Goldberg, & Cantú-Paz, 1998) which uses Bayesian networks to model promising solutions and subsequently guide the exploration of the search space.

In the BOA, the first population of strings is generated randomly with a uniform distribution. From the current population, the better strings are selected. Any selection method can be used. A Bayesian network that fits the selected set of strings is constructed. Any metric as a measure of quality of networks and any search algorithm can be used to search over the networks in order to maximize/minimize the value of the used metric. Besides the set of good solutions, prior information about the problem can be used in order to enhance the estimation and subsequently improve convergence. New strings are generated according to the joint distribution encoded by the constructed network. The new strings are added into the old population, replacing some of the old ones.

### 3 BAYESIAN NETWORKS

A Bayesian network (Pearl, 1988) is a directed acyclic graph with the nodes corresponding to the variables in the modeled data set (in our case, to the positions in solution strings). Mathematically, a Bayesian network encodes a joint probability distribution. A directed edge relates the variables so that in the encoded distribution, a variable corresponding to the terminal node is conditioned on a variable corresponding to the initial node. More incoming edges into a node result in a conditional probability of the corresponding variable with conjunctive condition containing all its parents. The network encodes independence assumptions that each variable is independent of any of its antecedents given its parents.

Various methods can be used to construct the network given the set of selected solutions. Most methods have two basic components: (1) a scoring metric which discriminates the networks according to their quality and (2) a search algorithm which searches over the networks to find the one with the best scoring metric value. The BOA can use any scoring metric and any search algorithm.

In our recent work we used a simple greedy algorithm to construct the network given the data. In each iteration of the algorithm, the graph operation that improves the network score the most is performed. The simple operations that can be performed on the network include edge additions, edge reversals, and edge removals. Only operations that keep the network acyclic are allowed and the number of parents of each node can be bound by a constant in order to avoid superfluously complex models. The construction finishes when no operations are allowed or no applicable graph operation improves the score.

The next two sections briefly discuss the Bayesian-Dirichlet and minimum description length metrics that can be used to evaluate competing networks.

#### 3.1 BAYESIAN-DIRICHLET METRIC

The Bayesian Dirichlet (BD) metric (Heckerman et al., 1994) combines the prior knowledge about the problem and the statistical data from a given data set. The probability of a Bayesian network  $B$  given data  $D$  can be computed by applying Bayes theorem as

$$p(B|D) = \frac{p(B)p(D|B)}{p(D)}. \quad (1)$$

The higher the  $p(B|D)$ , the more likely the network  $B$  is a correct model of the data. Therefore, the value of  $p(B|D)$  can be used to score different networks and measure their quality. This measure is called a Bayesian scoring metric, or the *posterior* probability of  $B$  given data  $D$ . Since we are only interested in comparing different networks (hypotheses) for a fixed data set  $D$ , we can eliminate the denominator  $p(D)$  of the above equation.

The probability  $p(B)$  is called the *prior* probability of the network  $B$  and it can be used to incorporate prior information about the problem by assigning higher probability to the networks confirming our intuition or expert knowledge. By using an empty prior network (with no edges) the metric favors simpler networks (see Section 4.2). Under a number of assumptions, the following closed expression can be derived

for  $p(D|B)$  (Heckerman et al., 1994):

$$p(D|B) = \prod_{i=0}^{n-1} \prod_{\pi_i} \frac{\Gamma(m'(\pi_i))}{\Gamma(m'(\pi_i) + m(\pi_i))} \prod_{x_i} \frac{\Gamma(m'(x_i, \pi_i) + m(x_i, \pi_i))}{\Gamma(m'(x_i, \pi_i))}, \quad (2)$$

where the product over  $\pi_i$  runs over all instances  $\pi_i$  of the parents  $\Pi_i$  of  $X_i$ , and the product over  $x_i$  runs over all instances  $x_i$  of  $X_i$ . By  $m(\pi_i)$ , the number of instances in  $D$  with  $\Pi_i$  instantiated to  $\pi_i$  is denoted. When the set  $\Pi_i$  is empty, there is one instance of  $\Pi_i$  and the number of instances with  $\Pi_i$  instantiated to this instance is set to  $N$  (the size of the data set  $D$ ). By  $m(x_i, \pi_i)$ , we denote the number of instances in  $D$  that have both  $X_i$  set to  $x_i$  as well as  $\Pi_i$  set to  $\pi_i$ . The metric computed according to the above equation is called the *Bayesian-Dirichlet metric*, since one of the assumptions made to compute the formula is that the parameters are distributed according to a Dirichlet distribution.

Terms  $m'(x_i, \pi_i)$  and  $m'(\pi_i)$  express our beliefs in frequencies  $m(x_i, \pi_i)$  and  $m(\pi_i)$ , respectively, and can be used as another source of prior information. A simple prior for the parameters  $m'(x_i, \pi_i)$  and  $m'(\pi_i)$  is to assume  $m'(x_i, \pi_i) = 1$  for all  $x_i$  and  $\pi_i$ , and compute  $m'(\pi_i)$  according to the above assignment. The metric using this assignment is called the K2 metric.

### 3.2 MINIMUM DESCRIPTION LENGTH METRIC

A minimum description length metric is based on the philosophical rule called Occam's razor, claiming that the simplest of competing theories be preferred to the more complex ones. The MDL metric favors short models in terms of their description length. A total description length of a data set  $D$  compressed according to a given model is defined as the sum of the space, measured in bits, required by the model, its parameters (various frequencies), and the data compressed according to the model. In context of evolutionary optimization the minimum description length was first time used by Harik (1999) in the extended compact genetic algorithm.

A directed acyclic graph can be encoded by storing a set of parents of each node. The set of parents of a particular node can be encoded by the number of the parents followed by the index of the set of parents in some agreed-upon enumeration of all possible subsets of variables of the corresponding cardinality. This results in  $\log_2 n + \log_2 \binom{n}{|\Pi_i|}$  bits for the parents of  $X_i$ .

To store the conditional probabilities according to the distribution encoded by the network, we need to store all combinations of all but one values  $x_i$  of each variable  $X_i$  and all possible instances  $\pi_i$  of its parents  $\Pi_i$ . For each such combination of  $x_i$  and  $\pi_i$  the corresponding conditional probability  $p(x_i|\pi_i)$  must be stored. For binary variables, there are  $2^{|\Pi_i|}$  possible combinations of values of the variable and its parents (excluding one value  $x_i$  for each  $\pi_i$ , e.g.  $x_i = 1$ , for which  $p(x_i|\pi_i)$  can be computed from the remaining conditional probabilities). This is an upper bound and can be reduced by using more sophisticated data structures to encode the conditional probability tables. To accurately encode each conditional probability, we can use  $\frac{1}{2} \log_2 N$  bits (Friedman & Yakhini, 1996). Thus, the overall number of bits needed to store the table of conditional probabilities for  $X_i$  is  $\log_2 N 2^{|\Pi_i|-1}$ .

The number of bits needed to store an instance with some probability is given by a logarithm of this probability. We must sum the description lengths over all individuals in the population. The total length of the model, its parameters, and the data set compressed according to this model is then given by the sum of the above terms (Pelikan, Goldberg, & Sastry, 2000).

A major advantage of the MDL metric is that it favors simple models so that no upper bound on the model complexity has to be specified. This bound comes up naturally. However, when using a greedy algorithm for model construction, the problem of finding a valid model can become more difficult. Moreover, the MDL metric does not easily permit the use of prior information about the problem. In many real-world problems the utilization of expert knowledge (which is often available in some form) may be unavoidable. Section 4.2 presents another way of dealing with the complexity of models by specifying the prior probability of each model inversely proportionally to its complexity.

A similar metric, called the Bayesian Information Criterion (BIC), was used in the EBNA (Etzeberria & Larrañaga, 1999) and the LFDA (Mühlenbein & Mahnig, 2000) algorithms.

## 4 DECISION GRAPHS IN BAYESIAN NETWORKS

Instead of encoding the conditional probability tables by a simple but inefficient probability table, one can use more sophisticated structures such as decision trees, decision graphs, and default tables (Chickering, Heckerman, & Meek, 1997; Friedman & Goldszmidt, 1999). In this fashion the number of parameters required to fully encode the distribution can be signif-

icantly decreased and the models become more expressive. This section explains how decision graphs can be used to improve the expressiveness of Bayesian networks and the learning of Bayesian network structure. It provides a metric for computing the likelihood of Bayesian networks with decision graphs and a method for constructing such structures (Chickering, Heckerman, & Meek, 1997). The network construction algorithm of Chickering et al. (1997) takes an advantage of using decision graphs by directly manipulating the network structure through the graphs. This is a major difference from the way the decision trees/graphs are usually used in context of the MDL metrics to reduce the description length (Friedman & Goldszmidt, 1999). Additionally, we provide the assignment of prior probabilities which takes into account model complexity.

#### 4.1 DECISION TREES AND GRAPHS

A decision tree is a directed acyclic graph where each node except for one designated node called the root has exactly one parent. The root has no parents. Non-leaf nodes of the tree are labeled by a variable (feature) on which we want to split. When a node is labeled by a variable  $v$ , we say that this node is a split on  $v$ . Edges from a split on  $v$  to its children (successors) are labeled by non-empty distinct exhaustive subsets of possible values of  $v$ .

To traverse a tree given an assignment of all the variables, we start in a root and on each split on  $v$  we continue to the child along the edge which contains the current value of  $v$ . Notice that for each instance (an assignment of all the variables) there exists only one possible way of traversing the tree to a leaf.

We will have one decision tree for each variable. Each leaf of this decision tree will contain conditional probabilities of different values of this variable given that the variables are constrained according to the path from the root to the leaf. An example adopted from Chickering et al. (1997) of a decision tree that encodes the conditional probability distribution  $p(z|x, y)$  is shown in Figure 1. All variables in this figure are binary and thus we can split only to two children, one for 0 and one for 1. Instance  $(x = 1, y = 1, z = 0)$  would traverse the tree to the right-most leaf. Instance  $(x = 0, y = 1, z = 0)$  would result in the middle leaf.

A decision graph is an extension of a decision tree in which each non-root node can have multiple parents. By a decision graph, any set of equality constraints among conditional probabilities can be encoded. This can be shown by simply constructing a complete tree and merging all leaves that are equal. An example

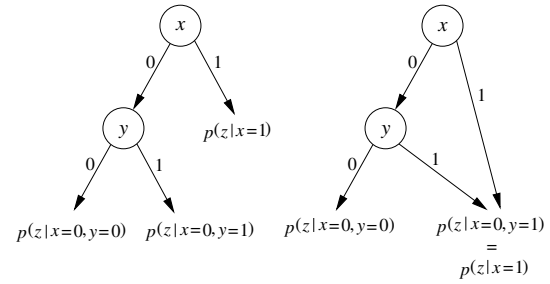


Figure 1: An example decision tree and graph encoding  $p(z|x, y)$ .

of a decision graph is shown in Figure 1. This decision graph can be obtained by merging the leaves  $p(z|x = 0, y = 1)$  and  $p(z|x = 1)$  which represents another equality constraint. It is important to note that the equality constraints, in fact, represent independence constraints. Moreover, each leaf in the decision graph for a variable represents independence assumptions of any variable not contained in the path from the root to this leaf, given the constraints specified by the corresponding path to this leaf. In fact, we do not need the Bayesian network anymore once we have the decision graphs. The network can be reconstructed by using the graphs.

There are four major advantages of using decision graphs in learning Bayesian networks. First, many fewer parameters can be used to represent a model. This saves memory and time requirements of both model construction as well as its utilization and allows representation of high-order relationships by reasonably-sized models. Second, the use of decision graphs allows learning a more complex class of models, because the relationships in a model can be cyclic under the constraint that different parts of the cycle are inconsistent with each other. It is beyond the scope of this paper to discuss this issue. Third, the construction of a Bayesian network with decision graphs performs smaller and more specific steps which may result in better models with respect to their likelihood. Finally, the network complexity measure can be easily incorporated into the scoring metric. The resulting measure allows the use of prior information unlike the MDL metric, and is as robust as the MDL metric when no such information is used. We will discuss this topic shortly.

## 4.2 BAYESIAN SCORE FOR NETWORKS WITH DECISION GRAPHS

In this section we briefly discuss the computation of a Bayesian score for Bayesian networks where conditional probabilities and independence assumptions for each variable are encoded by decision graphs (Chickering et al., 1997). This computation does not differ much from traditional Bayesian networks (see Equation 2). The outer product from Equation 2 remains the same. The middle product runs over all leaves of the decision graph  $G_i$  corresponding to the variable  $X_i$ . The inner-most product runs over all possible instances of the variable  $X_i$ . Thus,

$$p(D|B) = \prod_{i=0}^{n-1} \prod_{l \in L_i} \frac{\Gamma(m'(i, l))}{\Gamma(m(i, l) + m'(i, l))} \prod_{x_i} \frac{\Gamma(m(x_i, i, l) + m'(x_i, i, l))}{\Gamma(m'(x_i, i, l))}, \quad (3)$$

where  $L_i$  is the set of leaves in the decision graph  $G_i$  for  $X_i$ ,  $m(i, l)$  is the number of instances in  $D$  which end up the traversal through the graph  $G_i$  in the leaf  $l$ ,  $m(i, l)$  is the number of instances that have  $X_i = x_i$  and end up the traversal of the graph  $G_i$  in the leaf  $l$ , the  $m'(i, l)$  represents our prior knowledge about the value of  $m(i, l)$ , and  $m'(x_i, i, l)$  represents our prior knowledge about the value of  $m(x_i, i, l)$ .

To adjust the prior probability of each network according to its complexity, we first compute the description length of the parameters required by the networks. To encode one frequency in the data set of size  $N$ , it is sufficient to use  $0.5 \log_2 N$  bits (Friedman & Goldszmidt, 1999). Therefore, to encode all parameters, we need  $0.5 \log_2 N \sum_i |L_i|$  bits, where  $\sum_i |L_i|$  is the total number of leaves in all decision graphs. To favor simpler networks to the more complex ones, we can set the prior probability of a network to decrease exponentially with the description length of the set of parameters they require. Thus,

$$p(B) = c 2^{-0.5 \log_2 N \sum_i |L_i|}, \quad (4)$$

where  $c$  is a normalization constant required for the prior probabilities of all networks to sum to 1. The value of a normalization constant does not affect the result, since we are only interested in relative comparisons of networks and not the absolute value of their likelihood. A similar assignment of prior probabilities was presented in Friedman and Goldszmidt (1999). As we will see in the next section, the assignment in the last equation is sufficient to bias the model construction to networks with less parameters and avoid superfluously complex network structures

without having to determine the maximal number of incoming edges in advance. This eliminates another degree of freedom for setting the parameters of the algorithm and thus makes the algorithm easier to use. Somewhat weaker pressure toward simpler networks was introduced in Heckerman et al. (1994) and Chickering et al. (1997). Our experience was that the latter pressure was not strong enough to result in efficient learning in our application.

The above assignment can be extended or fully replaced by the one that takes into account our prior knowledge about the problem by favoring models that are more similar to the prior network.

## 4.3 LEARNING BAYESIAN NETWORKS WITH DECISION GRAPHS

To construct a decision graph on binary variables, two operators are sufficient. The first operator is a *split*, which splits a leaf on some variable and creates two new children of the leaf, connecting each of them with an edge associated with one possible value of this variable, in our case, 0 or 1. The second operator is a *merge*, which merges two leaves into a single leaf. It does not make sense to split a leaf on a variable that was encountered on the path from the root to this leaf and therefore these operators will not be allowed.

For variables that can obtain more than two values, two versions of the split operator can be considered: (1) a complete split which creates one child for each possible value of the variable (as above), and (2) a binary split, which creates one child corresponding to one particular value and another child for all the remaining values. Other alternatives can also be considered.

The greedy algorithms for constructing a Bayesian network using decision graphs differs from the one presented in Section 3 in that it does not manipulate the constructed network directly but only by modifying the decision graphs corresponding to each variable. The decision graph  $G_i$  for each variable  $X_i$  is initialized to a single-leaf graph, containing only probabilities  $p(X_i)$ .

In each iteration, all operators (e.g., all possible merges and splits) that can be performed on all decision graphs  $G_i$  are examined. The operator that improves the score the most is performed on the corresponding decision graph. Both split and merge operators can be performed. When making a split, we must make sure that no cycles appear in the network  $B$ . To guarantee that the final network remains acyclic, we can continuously update the network  $B$  each time we perform a split. Once we split a leaf of the graph  $G_i$  on a vari-

able  $X_j$ , we add an edge  $(X_j, X_i)$  to the network  $B$ . A more sophisticated algorithm is possible which would allow a cycle whose different parts would be incompatible and thus not form a true cycle. The pseudocode of the above algorithm follows.

- (1) Initialize a decision graph  $G_i$  for each node  $X_i$  to a graph containing only a single leaf.
- (2) Initialize the network  $B$  into an empty network.
- (3) Choose the best split or merge that does not result in a cycle in  $B$ .
- (4) If no improvement is possible, finish.
- (5) Execute the chosen operator.
- (6) If the operator was a split, update network  $B$ .
- (7) Go to (3).

It is important to notice the difference between the algorithm that directly modifies the network and the one which modifies the decision graphs. Adding an edge into a Bayesian network and using a full conditional probability table to store the corresponding probabilities corresponds to splitting *all* leaves of the decision graph corresponding to the terminal node of the edge on the variable corresponding to the initial node of the edge. However, by modifying only the decision graph, finer steps can be performed which may positively affect the quality of the resulting model.

## 5 EXPERIMENTS

This section starts by specifying our experiments. Subsequently, it provides and discusses the obtained results.

### 5.1 SPECIFICATION OF EXPERIMENTS

We have performed experiments on a number of functions. In this paper we only present results of our experiments on some of the functions. A simple linear function, called one-max, simply sums all bits. The 3-deceptive function is a sum of subfunctions of order 3, applied to disjoint blocks of 3 consecutive bits in the input string. Each of these subfunctions is defined as

$$f_{dec}^3(X) = \begin{cases} 0.9 & \text{if } u = 0 \\ 0.8 & \text{if } u = 1 \\ 0 & \text{if } u = 2 \\ 1 & \text{otherwise} \end{cases} \quad (5)$$

where  $X$  is a vector of 3 binary variables, and  $u$  is the sum of the input variables. The 3-deceptive function has one global optimum in  $(1, 1, \dots, 1)$  and  $2^{\frac{n}{3}}$  local optima in all points where  $(X_{3i} + X_{3i+1} + X_{3i+2}) \in \{000, 111\}$  for all  $i \in \{0, \dots, \frac{n}{3}\}$ . The above function is deceptive in a sense that an average value over all strings that contain 0's in two particular positions is

greater than the corresponding average with the 1's in these two positions. This feature makes the function very difficult unless the algorithm uses a recombination that respects interactions.

The two dimensional Ising spin-glass function maps bits onto a regular 2D grid. There are two types of edges. An edge of the first type contributes to the overall fitness by 1 if the bits at its ends are the same. An edge of the second type contributes to the fitness if the bits at its ends are different. For a more detailed description of the function and the definition of the problem instance we used in our experiments, please see Pelikan et al. (1998).

We have compared the simple GA, the BOA with both the MDL metric as well as the Bayesian-Dirichlet metric, and the BOA using decision graphs to construct the model and encode its parameters. The complexity measure was incorporated into the metric for the use with decision graphs as described in the above sections. The performance of the BOA does not depend on how we order the variables in solution strings. On the other hand, the ordering of the variables strongly influences the performance of the simple GA. We have chosen the representation so that interacting variables are close in the solutions strings. This is the best case for the simple GA. However, the main purpose of our experiments was not to compare the BOA to the simple GA but to give insight in the BOA and the effect of using local structures on its performance.

The population size in each algorithm was set as the minimal population size required for the algorithm to converge in all of 30 independent runs. Binary tournament selection was used in all experiments. The number of offspring is equal to a half of the population size and the generated offspring replace the worst half of the original population. In this fashion, the runs are more stable and elitism is introduced. The probability of crossover in the simple GA was determined according to empirical evidence presented elsewhere (Pelikan, Goldberg, & Cantú-Paz, 1999) as  $p_c = 1$ . Except for the one-max function, the mutation was not used, since it seems to not pay off on the tested problems. On one-max problem,  $p_m = 0.01$  was used.

### 5.2 RESULTS

The results on the simple linear one-max function are shown in Figure 2. One-max is a linear function and thus it is expectable that all the algorithms perform very well. The BOA with an optimal  $k = 0$  (which is equivalent to the UMDA) performs the best, because it does not take into account any interactions and in this problem all the variables are indeed independent.

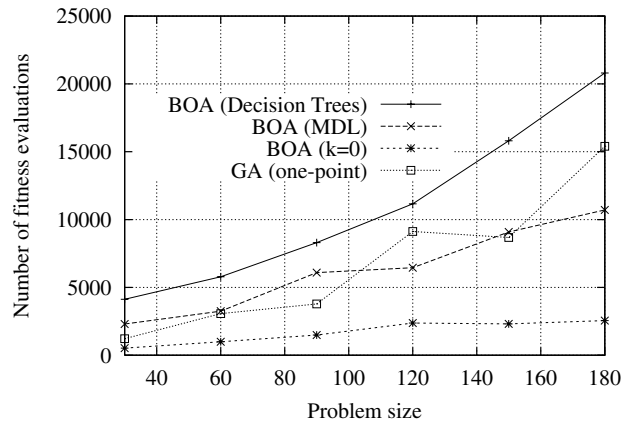


Figure 2: Results on the one-max function.

Other algorithms try to model some interactions and that is why they perform worse. However, all algorithms seem to scale very well. Using decision graphs does not seem to pay off in this case because of that the noise misleads the model building which is very sensitive when the decision graphs are used. Other modeling techniques make bigger steps and it is harder to mislead them. Even though this feature is likely to make the BOA work less efficiently on this simple linear problem, it can be very useful when solving difficult problems where making little steps while building the model pays off.

The results on the 3-deceptive function are shown in Figure 3. The results suggest that all versions of the BOA perform similarly and as the problem size grows, they outperform the simple GA with one-point crossover. This suggests that they scale up better. It is important to note that the coding chosen for this problem is the best one for the simple GA. If the building blocks (the bits corresponding to each deceptive subfunction) would be coded more loosely, the performance of the simple GA would get worse, and, eventually, for a random ordering of the variables in the strings representing solutions the simple GA would require exponential time (Thierens & Goldberg, 1993). However, the BOA is independent of the ordering of the bits in strings, and thus its performance would remain the same independently of the ordering we choose. The BOA with both the decision graphs and the MDL metric performs very similarly. This behavior is very interesting because the metrics are coming from two different paradigms.

The results on the spin-glass system are provided in Table 5.2. The BOA with the MDL metric performs the best. The BOA with  $k = 2$  performs the worst. The simple GA does not reach the optimum even with huge populations and a large number of generations.

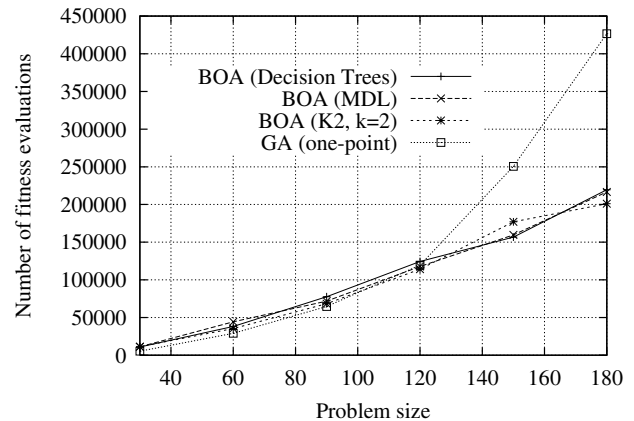


Figure 3: Results on the 3-deceptive function.

This problem is very difficult for the simple GA. It is interesting that the MDL metric performs the best and outperforms both the BOA with decision graphs as well as a bound on the network complexity  $k = 2$  and the K2 metric. We have also performed experiments with higher values of  $k$ . As the  $k$  increases to 4, the performance of the BOA with the K2 metric improves. The BOA with  $k = 4$  and the K2 metric performs the best of all compared algorithms. However, with  $k = 5$  the performance again decreases. This suggests that the value of  $k = 4$  is a good choice for this problem.

Table 1: Results on the 2D spin-glass.

| Algorithm         | Fitness evals | Std. dev. |
|-------------------|---------------|-----------|
| BOA ( $k=2$ , K2) | 75833.33      | 4649.03   |
| BOA ( $k=3$ , K2) | 42733.33      | 2993.48   |
| BOA ( $k=4$ , K2) | 36606.67      | 2045.04   |
| BOA ( $k=5$ , K2) | 48960.00      | 2099.85   |
| BOA (MDL)         | 38091.67      | 2317.69   |
| BOA (Dec. trees)  | 57960.00      | 3109.62   |

## 6 CONCLUSIONS

The use of local structures to represent conditional probability tables has four major advantages. First, the number of parameters required to store probabilities with a large conditional part can decrease significantly. This makes the method work more efficiently as we increase the complexity of models. Moreover, high-order interactions can be represented by using models of reasonable size. Second, by using decision graphs to guide the network construction, one can discover more complicated relationships which may not be evident when directly modifying the network. Additionally, models of the same quality can be discovered by using less effort. Third, the models become more expressive, since cycles with different parts corresponding to in-

compatible paths in the decision graphs may appear in the model.

Finally, the complexity of the models can be automatically controlled by making prior probabilities of competing models be inversely proportional to their complexity. Our experiments suggest that setting the prior probability of a network to be inversely proportional to the number of bits required to store the frequencies in the network works well. By using Bayesian scoring metric containing a complexity measure as described above, one can both (1) use prior knowledge about the problem in network construction and (2) eliminate the need for a bound on the network complexity. In this fashion one can get the best of the two approaches.

Our test problems do not benefit from the compact representation by local structures, such as decision graphs. They only require covering interactions of a bounded order. Hot candidates for the use of local structures are problems that require a hierarchical approach where we must encode interactions of a very high order with a quite regular and simple structure. Such interactions may appear later in the run and require efficient representation.

## ACKNOWLEDGMENTS

The authors would like to thank David Heckerman, David Chickering, Martin V. Butz, Erick Cantú-Paz, Josef Schwarz, and Jiri Ocenasek for useful comments and help with the project.

The work was sponsored by the Air Force Office of Scientific Research, Air Force Materiel Command, USAF, under a grant F49620-00-1-0163. Research funding for this work was also provided by a grant from the National Science Foundation under grant DMI-9908252. Support was also provided by a grant from the U. S. Army Research Laboratory under the Federated Laboratory Program, Cooperative Agreement DAAL01-96-2-0003. The US Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation thereon. Martin Pelikan was partially supported by grant VEGA 1/7654/20 of Slovak Grant Agency.

The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of the Air Force Office of Scientific Research, the National Science Foundation, the U. S. Army, or the U.S. Government.

## References

- Bosman, P. A., & Thierens, D. (2000). Continuous iterated density estimation evolutionary algorithms within the IDEA framework. In Wu, A. (Ed.), *Workshop proceedings of the Genetic and Evolutionary Computation Conference GECCO 2000* (pp. 197–200). San Francisco, CA: Morgan Kaufmann.
- Chickering, D. M., Heckerman, D., & Meek, C. (1997). *A Bayesian approach to learning Bayesian networks with local structure* (Technical Report MSR-TR-97-07). Redmond, WA: Microsoft Research.
- Etzeberria, R., & Larrañaga, P. (1999, March). Global optimization using Bayesian networks. In *Second Symposium on Artificial Intelligence (CIMA-99)* (pp. 332–339). Habana, Cuba.
- Friedman, N., & Goldszmidt, M. (1999). Learning Bayesian networks with local structure. In Jordan, M. I. (Ed.), *Graphical models* (1 ed.). (pp. 421–459). Cambridge, MA: MIT Press.
- Friedman, N., & Yakhini, Z. (1996). On the sample complexity of learning Bayesian networks. In Horvitz, E., & Jensen, F. (Eds.), *Proceedings of the 12th Conference on Uncertainty in Artificial Intelligence (UAI-96)* (pp. 274–282). San Francisco: Morgan Kaufmann Publishers.
- Harik, G. (1999). *Linkage learning via probabilistic modeling in the ECGA* (IlliGAL Report No. 99010). Urbana, IL: University of Illinois at Urbana-Champaign, Illinois Genetic Algorithms Laboratory.
- Heckerman, D., Geiger, D., & Chickering, M. (1994). *Learning Bayesian networks: The combination of knowledge and statistical data* (Technical Report MSR-TR-94-09). Redmond, WA: Microsoft Research.
- Mühlenbein, H., & Mahnig, T. (2000). Evolutionary algorithms using marginal distributions. *New Generation Computing*, 18, 157–166.
- Mühlenbein, H., & Paaß, G. (1996). From recombination of genes to the estimation of distributions I. Binary parameters. In Eiben, A., et al. (Eds.), *Parallel Problem Solving from Nature - PPSN IV* (pp. 178–187). Berlin: Springer Verlag.
- Pearl, J. (1988). *Probabilistic reasoning in intelligent systems: Networks of plausible inference*. San Mateo, California: Morgan Kaufmann.
- Pelikan, M., Goldberg, D. E., & Cantú-Paz, E. (1998). *Linkage problem, distribution estimation, and Bayesian networks* (IlliGAL Report No. 98013). Urbana, IL: University of Illinois at Urbana-Champaign, Illinois Genetic Algorithms Laboratory.
- Pelikan, M., Goldberg, D. E., & Cantú-Paz, E. (1999). BOA: The Bayesian optimization algorithm. In Banzhaf, W., et al. (Eds.), *Proceedings of the Genetic and Evolutionary Computation Conference GECCO-99*, Volume I (pp. 525–532). San Francisco, CA: Morgan Kaufmann.
- Pelikan, M., Goldberg, D. E., & Lobo, F. (2000). A survey of optimization by building and using probabilistic models. *Computational Optimization and Applications*. In press.
- Pelikan, M., Goldberg, D. E., & Sastry, K. (2000). *Bayesian optimization algorithm, decision graphs, and Occam's razor* (IlliGAL Report No. 2000020). Urbana, IL: University of Illinois at Urbana-Champaign, Illinois Genetic Algorithms Laboratory.
- Thierens, D., & Goldberg, D. E. (1993). Mixing in genetic algorithms. In Forrest, S. (Ed.), *Proceedings of the Fifth International Conference on Genetic Algorithms* (pp. 38–45). San Mateo, CA: Morgan Kaufmann.